Joint Model and Data Adaptation for Cloud Inference Serving

Jingyan Jiang*†, Ziyue Luo‡, Chenghao Hu§, Zhaoliang He*, Zhi Wang*¶, Shutao Xia*¶, Chuan Wu‡

*Tsinghua University, †Sangfor Technologies, ¶Peng Cheng Laboratory
‡The University of Hong Kong, §University of Toronto
Email: jiangjingyan@tsinghua.edu.cn, hezl19@mails.tsinghua.edu.cn
{wangzhi, xiast}@sz.tsinghua.edu.cn, {zyluo, cwu}@cs.hku.hk, ch.hu@mail.utoronto.ca

***Abstract*—Real-time deep learning inference serving systems often require prohibitive resources and diverse user requirements. The existing design of inference serving systems mainly focusing on computation resource efficiency, largely ignoring the trade-off between computation and bandwidth resources in need. Sub-optimal resource utilization usually leads to huge serving cost waste. In this paper, we tackle the dual challenge of computation-bandwidth trade-off and cost-effectiveness by proposing A^2, an efficient joint Adaptive model, and Adaptive data deep learning serving solution across the geo-datacenters. Inspired by the insight that a trade-off between computational cost and bandwidth cost in achieving the same accuracy, we design a real-time inference serving framework, which selectively places different "versions" of the deep learning models at different geo-locations, and schedules different data sample versions to be sent to those model versions for inference. The goal is to minimize the total serving cost while meeting latency and accuracy demand for the serving requests. We formulate a joint placement and serving problem and propose an efficient approximation algorithm to solve it with a theoretical performance guarantee. We deploy A^2 on Amazon EC2 for experiments, which shows that A^2 achieves 30%-50% serving cost reduction under the same required latency and accuracy as compared to baselines.**

***Index Terms*—Deep Model Serving, Model Compression, Data Compression, Resource Allocation**

I. Introduction

Inference serving (or *prediction serving*) is a real-time service to return inference results (*i.e.,* predictions) upon receiving requests carrying data samples, by passing them through pre-trained deep machine learning models (*i.e.,* inference). Machine learning has been successful in powering various applications nowadays, and various prediction serving systems have been proposed by both cloud vendors [1]–[3] and open-source communities [4]–[7]. For example, Facebook serves simultaneously over 200 *trillion* requests worldwide [8].

A key issue in cloud inference serving systems is to efficiently deploy the machine learning models across a geographic span, and provide resources to meet the latency and accuracy demand of inference queries, in a cost-effective manner. Inference serving is typically resource-intensive: the inference execution is computation costly (*e.g.,* the floating point operations (FLOPs) per inference can be up to 192.2 TFLOPs for BERT-Large [9]) and bandwidth costly (*e.g.,* some tasks require high-resolution images to be uploaded). Thus the inference serving cost can be a challenge if there are huge volumes of queries in the inference serving system [10].

Several studies have been devoted to improving the resource utilization efficiency in a model inference system. Zhang et al. [11] propose *VideoStorm* to handle video analysis requests using deep learning models, and carry out specific configuration to satisfy a user utility based on different requirements of accuracy and latency. Based on the fact that different inference queries generally have different requirements on response latency and prediction accuracy [10], Halpern et al. [7] propose a framework called OSFA, which allows users to choose their tolerance tiers so that they can sacrifice inference quality by using multiple model versions that may differ in inference latency and accuracy, to improve other aspects of the service, such as the service response time.

However, previous studies only focus on the efficient utilization of computational resources and ignore the trade-off between *bandwidth* and *computation* resources in the cloud inference systems. In this paper, we perform an in-depth measurement to show that the inference accuracy can be affected by both input sample size and model complexity. This reveals a trade-off between computational cost and bandwidth cost: different combinations of input data and model versions can yield similar latency and accuracy metrics. It allows us a larger optimization space for cloud model deployment and request scheduling.

Motivated by these observations, we propose A^2, which jointly adapts the model and data versions to serve incoming inference queries, to satisfy their diverse latency and accuracy requirements, with minimized overall serving cost. As illustrated in Fig. 1, we deploy serving models of different versions in geo-distributed data centers and schedule the queries accordingly. The contributions of our study are summarized as follows.

▷ We investigate important factors affecting accuracy and latency in deep learning serving. Our studies based on deep model compression [12], [13] show that input data down-

This work is supported in part by the National Key Research and Development Program of China under Grant 2018YFB1800204. Also, this work is supported in part by NSFC (Grant No. 61872215), and Shenzhen Science and Technology Program (Grant No. RCYX20200714114523079) and Hong Kong RGC under the contracts HKU 17204619, 17208920. Zhi Wang is the corresponding author.

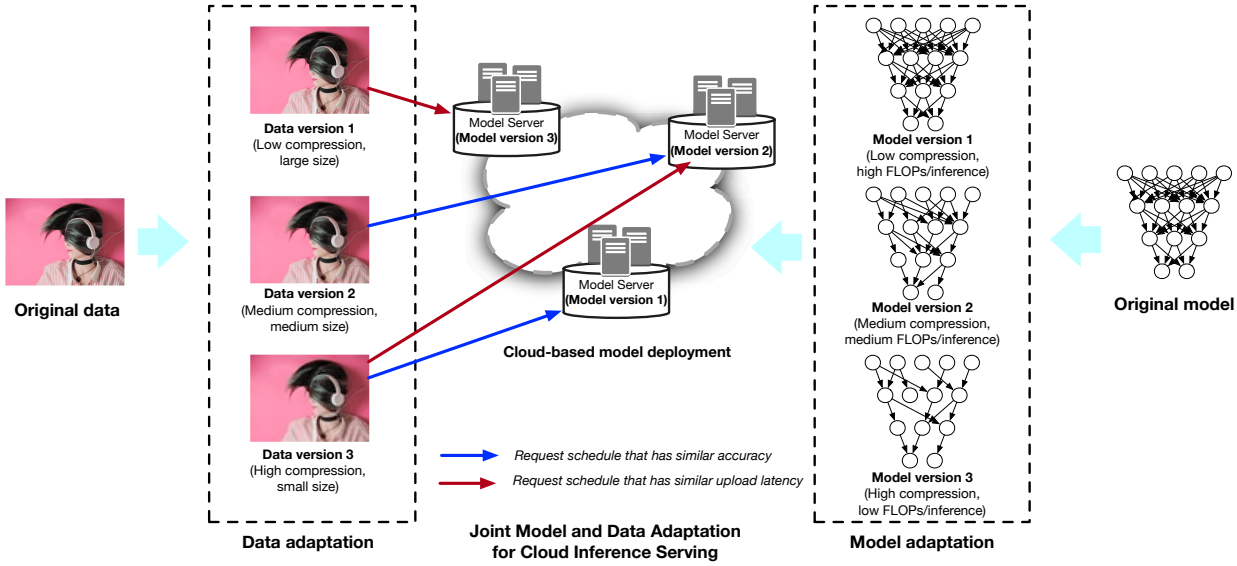


Fig. 1: The concept architecture of joint model and data adaptation for cloud inference serving[1].

sampling and model compression can both affect inference quality: a larger input sample size usually leads to more significant upload latency and higher inference accuracy, while a larger model size implies larger inference latency and higher accuracy. As illustrated in Fig. 1, the same latency and accuracy requirement can be satisfied by different model deployment and request schedules. Hence, trade-offs exist in achieving similar latency and accuracy with different amounts/types of resources. The deep model inference service should jointly consider data adaptation and model adaptation for better resource utilization.

▷ Inference serving with multiple model versions and data versions leads to a much larger optimization space as compared to conventional placement problems [14], [15]. Thus, we formulate the model deployment and query serving problem as an integer linear program, which minimizes the system cost while satisfying latency and accuracy constraints of the queries. We propose an approximation algorithm to solve the problem in polynomial time, which yields fractional solutions; Then, our algorithm rounds the fractional results without latency violation for requests, i.e., it picks the datacenters for model deployment from the *intersection* of ones actually satisfying the latency and accuracy constraints.

▷ We deploy and evaluate A^2 on AWS EC2, with real-world deep learning models and query traces. Our experiment results are as follows. 1) Our joint adaptive model and data design reduces the inference serving cost by 50%, as compared to baselines including a fixed scheme (same as Clipper [5] and Clockwork [16]), and a naive model and data-adaptive inference scheme (same as INFaaS [6], OSFA [7] and AWStream [17]). 2) In particular, our proposed rounding algorithm reduces the model deployment cost by 30%, as compared to other heuristic rounding schemes. 3) Our method dynamically trades off bandwidth and computation resources while satisfying the accuracy and latency constraints.

II. Background and Motivation

A. Model and Data Adaptation affects the upload latency and accuracy

In machine learning driven services/applications (e.g., computer vision oriented), the input data (e.g., an image) can be resized and/or compressed into different *versions*. Thanks to model compression technologies, deep learning models can also be "compressed" into versions with different parameter scales. To understand the impact of model and data adaptation on the accuracy and computational resource consumption, we carry out measurement studies on representative models, including ResNet18 and MobileNetV1. In our measurement, we use model pruning that selectively removes non-significant filters, weight parameters, and channels, to compress a deep learning model. In particular, based on *PocketFlow* [13], an open-source automatic model compression (AutoMC) framework to compress the selected models, we use *discrimination aware channel pruning*[2] to compress an original model, into the four *model versions* that differ in model complexity. We define the original model version as model version 0, and model version 1 (2 and 3, respectively) with pruned ratio[3] $\rho = 40\%$ (60%, 80%, respectively)

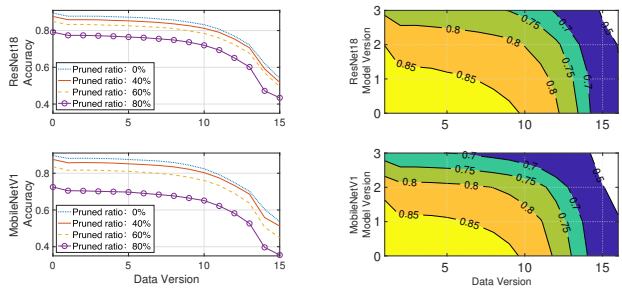
For input *data version* which differs in data size, we randomly selected $5,000$ images from ImageNet and resize each image size from 224×224 down to 32×32, with step size twelve to obtain sixteen different versions saved as .PNG files. The average data size degrades from 92KB (data version 0: the original data size) to 5KB (data version 15: the smallest data size). Next, we present the measurement results.

First, we observe from Fig. 2(a) that the accuracy slightly decreases under pruning: only 2% accuracy deduction with

[1]The original face figure referred to: https://demo.mysterythemes.com/child-theme/easy-shop/product/creative-free-online-store-theme/

[2]It introduces a group of additional discriminative losses into the model and preserve the channels helpful to discriminative power.

[3]To drop ρ FLOPs of the original model.



(a) Top-5 Validation Accuracy (b) Accuracy Contour

Fig. 2: Measurement results. (a) is the Top-5 validation accuracy of each model versions feeded by input data with each data version. (b) is the "contour plots of accuracy". The curves joining points at the accuracy have the same value.

40% pruned. On the other hand, the accuracy drops more significantly with the data size reduction. Based on our measurement study on .JPEG image compression, it also demonstrates similar accuracy versus data size patterns.

Second, to achieve the same accuracy level (AL)[4] for a deep learning model, one can choose different combinations of data versions and model versions, according to Fig. 2(b). For example, for ResNet18, (data version 5, model version 2) and (data version 10, model version 0) have the same accuracy level of 80%.

Besides, the data size will affect the data upload latency especially when bandwidth is scarce or original input data size is large. For instance, when the upload bandwidth is 400KB/s, the upload latency will reduce from $230ms$ to $12.5ms$ if we use data version 15.

B. Diverse Requirements of Requests in latency and accuracy

While many requests in deep model inference services requiring results in real-time, others can tolerate end-to-end latency of even many minutes; accuracy requirements of these application requests have a great distance as well [10], [11], [18]. For instance, the requests for counting cars to control the traffic is served with strict latency requirement, but the approximate car counts could be accepted. In contrast, the requests for license plate recognizing application could tolerate with even many minutes because the billing can be delayed, but can not tolerate lower prediction accuracy. In addition, recommendations and serving advertisements require high prediction accuracy and low response delay at the same time.

Takeaways: *Based on these insights, the joint model and data adaptation introduces a new design space by trade-off computation consumption and bandwidth consumption when configuring inference services.*

III. Problem Model

A. Machine Learning Inference Service Model

We consider a machine learning inference service provider who rents VMs from S data centers distributed in S distinct geographical regions to host machine learning models. Without loss of generality, we focus on the deployment of the machine learning model in a time window, Δt.

The model serves machine learning requests across the S regions, where each machine learning request is a demand from the end-user requiring the model to make inference based on one input data, *e.g.,* classifying one picture. For each machine learning request, the end-user specifies a quality requirement for latency and accuracy within the available range provided by the system. For ease of presentation, we group the machine learning requests originating from one geographical region during Δt with the same quality requirement, and refer to the request group as *one request* hereinafter. There are Q requests in total, and we use Y^q to signify the number of machine learning requests in request $q \in [Q]$[5]. We denote the accuracy requirement and latency requirement of request q as α^q and β^q, respectively.

The model is compressed into into I model versions, and the input data will be compressed into J data versions. The data size of data version $j \in [J]$ is represented as b_j. We use j_{max} to represent the data version with original data size. Given a model version i and a data version j, the serving accuracy a^q_{ij} for a request q is known. We use A^q to represent the set of all 2-tuple $< i, j >$ that satisfy the accuracy requirement of q, *i.e.,* $A^q = \{< i, j > | i \in [I], j \in [J], a^q_{ij} \geq \alpha^q\}$.

One container or VM instance loads one model file in a certain version and is referred to as one *model server*. The monetary cost for running one model server with version i (model-server-i)[6] during the time window is c_i. The bandwidth of model-server-i in s is denoted as B_{is}. The machine learning requests could be aggregated into a batch for processing in the data center. We denote the set of possible batch size as H. However, if the model inference leverages CPU rather than GPU, we set $H = \{1\}$. Moreover, the batches per second (BPS) for model-server-i with batch size $h \in H$ is denoted as $V_i(h)$.

The end-to-end latency for a request handling by model-server-i deployed on data center s using data version j and batch size h is represented as $l^q_{ijs}(h)$. The end-to-end latency considering GPU-based model consists of four parts: the transmission latency l_{trans}, the propagation latency l_{prop}, the requests batching latencyl_{batch} and the inference latency l_{infer}. We ignore the receiving latency since the result data is too small compared to the input data. We detail the formulation for each component below. For CPU-based model, we ignore l_{batch} and set h to 1.

1) The transmission latency is the amount of time required to send all the data into the wire, which can be formulated as follows:

$$l_{trans} = \frac{b_j}{B^q_s} \tag{1}$$

where B^q_s is the transmission rate from request q to data center s.

[4]The accuracy provided is no lower than AL.

[5]We use $[X]$ to represent the set $\{1, 2, ..., X\}$ in the paper.

[6]We use model-server-i for short in the following of the paper

TABLE I: NOTATION

S	# of data centers
Q	# of requests
I	# of model versions
J	# of data versions
Y^q	# of machine learning requests in request q
A^q	set of all $< i, j >$ satisfying the accuracy requirement of q
$S^q_{ij}(h)$	set of all data centers satisfying the latency requirement of q using model version i with data version j and batch size h
B_{is}	bandwidth capacity per model-server-i in data center s
H	set of possible batch sizes
$V_i(h)$	BPS for model-server-i with batch size h
c_i	monetary cost for running one model-server-i during Δt
Δt	time window length
$x_{is}(h)$	# of model-server-i deployed onto data center s with batch size h
$y^q_{ijs}(h)$	the proportion of request q that is sent to model-server-i on s with data version j and batch size h

2) The propagation latency is the amount of time for sending each packet across WAN to the destination. We denote the propagation latency between q and s as d^q_s. Thus, We formulate the propagation latency as:

$$l_{prop} = d^q_s \tag{2}$$

3) The requests batching latency refers to the time for batching H_i requests:

$$l_{batch} = \frac{h\Delta t}{R_{is}} \tag{3}$$

where R_{is} is the average number of machine learning requests processed by one model-server-i on data center s during Δt. However, as we assume the requests are intensive, we can use the maximal number of machine learning requests processed by one model-server-i as an estimation for R_{is}, which is either $V_i(h)h\Delta t$ or $B_{is}/b_{j_{max}}$ if the bandwidth is saturated. Therefore, we can estimate the requests batching latency as:

$$l_{batch} = \frac{h\Delta t}{\min\{V_i(h)h\Delta t, B_{is}/b_{j_{max}}\}} \tag{4}$$

4) The inference latency is the time for one inference:

$$l_{infer} = \frac{1}{V_i(h)} \tag{5}$$

Leveraging Eqn. (1), (2), (4) and (5), we have:

$$l^q_{ijs} = \frac{b_j}{B^q_s} + d^q_s + \frac{h\Delta t}{\min\{V_i(h)h\Delta t, B_{is}/b_{j_{max}}\}} + \frac{1}{V_i(h)} \tag{6}$$

Therefore, for each 4-tuple $< i, j, h, q >$, we can find a subset of S whose every element satisfies the latency requirement β^q if we deploy model-server-i onto it serving request q with data version j and batch size h. We denote such a subset as $S^q_{ij}(h)$.

Important notations are listed in Table I.

B. Decision Variables

Two types of optimization variables are formulated: 1) $x_{is}(h)$, the number of model-server-i deployed onto data center s with batch size h; and 2) $y^q_{ijs}(h)$, the proportion of request q that is sent to model-server-is on data center s and is compressed into data version j with batch size h.

C. Problem Formulation

Our goal is to minimize the overall *Deployment Cost*, C_{all}, of the whole machine learning inference service system while serving all the requests with individual quality requirements. The overall cost of deploying model servers can be derived as follows:

$$C_{all} = \sum_{i\in[I]}\sum_{h\in H}\sum_{s\in[S]} c_i x_{is}(h) \tag{7}$$

We can formulate the cost minimization problem as an integer linear programming (ILP) as follows:

$$\textbf{min} \sum_{i\in[I]}\sum_{h\in H}\sum_{s\in[S]} c_i x_{is}(h) \tag{8}$$

subject to:

$$\sum_{i\in[I]}\sum_{j\in[J]}\sum_{h\in H}\sum_{s\in[S]} y^q_{ijs}(h) = 1, \forall q \in [Q] \tag{8a}$$

$$\sum_{q\in[Q]}\sum_{j\in[J]} Y^q y^q_{ijs}(h) \le \Delta t V_i(h) h x_{is}(h), \forall i \in [I], h \in H, s \in [S] \tag{8b}$$

$$\sum_{q\in[Q]}\sum_{j\in[J]} Y^q b_j y^q_{ijs}(h) \le B_{is} x_{is}(h), \forall i \in [I], h \in H, s \in [S] \tag{8c}$$

$$y^q_{ijs}(h) = 0, \forall q \in [Q], i \in [I], j \in [J], h \in H, s \notin S^q_{ij}(h) \tag{8d}$$

$$y^q_{ijs}(h) = 0, \forall q \in [Q], h \in H, s \in [S], < i, j > \notin A^q \tag{8e}$$

$$x_{is}(h) \in \{0, 1, 2, \ldots\}, \forall i \in [I], h \in H, s \in [S] \tag{8f}$$

$$y^q_{ijs}(h) \ge 0, \forall q \in [Q], i \in [I], j \in [J], h \in H, s \in [S] \tag{8g}$$

Constraint (8a) ensures that every request is served. Constraint (8b) and constraint (8c) together guarantee that we deploy enough model servers to serve all the requests, while both the process capacity and bandwidth of every model server are not violated. The quality requirements are respected due to the latency constraint (8d) and the accuracy constraint (8e).

The cost minimization problem is then a mixed integer linear program (MILP).

IV. A Rounding-based Approximation Algorithm

A. The Rounding Scheme

We present our rounding-based approximation (RA) algorithm for the MILP problem (8) in this section. Our approximation algorithm contains two stages: in the *first stage*, we solve the original MILP as a linear program (LP) to obtain a fractional solution; and in the *second stage*, we round the fractional solution to a feasible integer solution that satisfies problem (8).

Algorithm 1 Set Construction Algorithm - SC

Input: $Q, \{\hat{y}^q_{i'j's}(h')\}_{\forall q,s:\hat{x}_{is}(h) \text{ is fractional}}$
Output: $\Theta, \Theta_{all}, \{z^q_{i'j's}(h')\}_{\forall q,s:\hat{x}_{is}(h) \text{ is fractional}}$

Calculate $z^q_{i'j's}(h'), \forall q \in [Q]$ based on Eqn. (9)
$\Theta_{all} \leftarrow \varnothing$
$\Theta \leftarrow \{\}$
for $q \in [Q]$ **do**
  $\Theta^q \leftarrow S^q_{ij}(h)$
  if $\exists s \in \Theta^q : z^q_{i'j's}(h') > 0$ **then**
    Add Θ^q to Θ_{all}
  end if
end for
$\Theta^{copy}_{all} \leftarrow \Theta_{all}$
while $\Theta_{all} \neq \varnothing$ **do**
  $s_{max} \leftarrow \arg\max_{s\in[S]} |\{\Theta^q \in \Theta_{all} | s \in \Theta^q\}|$
  $\Theta_{all} \leftarrow \Theta_{all} \setminus \{\Theta^q \in \Theta_{all} | s_{max} \in \Theta^q\}$
  Θ.append(s_{max})
end while
return Θ, Θ^{copy}_{all}, $\{z^q_{i'j's}(h')\}_{\forall q,s}$

In the *first stage*, we relax constraint (8f) to $x_{is}(h) \geq 0$. Therefore, we transform problem (8) to an LP. Leveraging the Interior Point method [19], we can solve the LP in polynomial time and obtain a fraction solution to the original problem (8) denoted by $\hat{x}_{is}(h)$ and $\hat{y}^q_{ijs}(h)$.

In the *second stage*, we round $\hat{x}_{is}(h)$ to an integer solution $\bar{x}_{is}(h)$ and modify $\hat{y}^q_{ijs}(h)$ accordingly while ensure all constraints in (8) are respected. For ease of presentation, we illustrate the rounding process considering a specific model version i' with data version j' and batch size h'. We use real value $z^q_{ijs}(h)$ to denote the number of model-server-i with data version j and batch size h on s used to process request q. For each $\hat{x}_{i's}(h')$, either the corresponding constraint (8b) or (8c) is tight. Therefore, we calculate $z^q_{i'j's}(h')$ as follows:

$$z^q_{i'j's}(h') = \begin{cases} \frac{Y^q y^q_{i'j's}(h')}{\Delta t V_{i'}(h')h'} & \text{if (8b) is tight} \\ \frac{Y^q b^q_{j'} y^q_{i'j's}(h')}{B_{i's}} & \text{if (8c) is tight} \end{cases} \tag{9}$$

For each request q satisfying that at least one corresponding $z^q_{i'j's}(h')$ is greater then 0, we define $\Theta^q = S^q_{i'j'}(h')$. We want to construct a subset of S denoted as Θ ensuring $\Theta \cap \Theta^q \neq \varnothing, \forall \Theta^q$ and the size of Θ should be as small as possible. We design a set construction (SC) algorithm to establish Θ for $< i', j', h' >$ as described in Alg. 1. We first find all the Θ^q satisfying $\exists s \in \Theta^q : z^q_{i'j's}(h') > 0$ and group them in both Θ_{all} and Θ^{copy}_{all}. Then, we iteratively add the s_{max} satisfying $\arg\max_{s\in[S]} |\{\Theta^q \in \Theta_{all} | s \in \Theta^q\}|$ to Θ until Θ satisfies the above requirement.

Lemma 1. *$|\Theta|$ constructed in Alg. 1 is at most $\frac{|N|}{R}\ln|N|$, where $N = \bigcup_{q:\Theta^q\in\Theta_{all}} \Theta^q$ and $R = \min\{|\Theta^q| | \Theta^q \in \Theta_{all}\}$.*

Proof. We use s_k to denote the k-th s that is added to Θ. And we use T_k to denote $|\Theta_{all}|$ after s_k has been added to Θ. In addition, use $g(s)$ to denote $|\{\Theta^q \in \Theta_{all} | s \in \Theta^q\}|$.

Algorithm 2 Rounding-based Approximation Algorithm - RA

Input: $S, Q, I, J, H, \Delta t, \{Y^q\}_{\forall q}, \{A^q\}_{\forall q}, \{S^q_{ij}(h)\}_{\forall q,i,j,h}, \{B_{is}\}_{i,s}, \{V_i(h)\}_{\forall i,h}, \{c_i\}_{\forall i}$
Output: $\{\bar{x}_{is}(h)\}_{\forall i,s,h}$, $\{\bar{y}^q_{ijs}(h)\}_{\forall q,i,j,s,h}$

1: Solve the relaxed LP (8) using interior point method.
2: Obtain the optimal fraction solution $\{\hat{x}_{is}(h)\}$ and $\{\hat{y}^q_{ijs}(h)\}$
3: **for** $i \in [I]$, $j \in [J]$, $h \in H$ **do**
4: $\Omega \leftarrow \{\hat{y}^q_{ijs}(h), \forall q \in [Q], s \in [S] | \hat{x}_{is}(h) \text{ is fractional}\}$
5: $\Theta, \Theta_{all}, \{z^q_{ijs}(h)\} \leftarrow$SC$(Q, \Omega)$
6: **for** $q \in \{q \in [Q] | \Theta^q \in \Theta_{all}\}$ **do**
7: Select the first s in Θ that exists in Θ^q as $\hat{s}$
8: **for** $s \in \Theta^q \setminus \hat{s}$ **do**
9: **if** $z^q_{ij\hat{s}}(h) - \lfloor z^q_{ij\hat{s}}(h) \rfloor \leq \lceil z^q_{ijs}(h) \rceil - z^q_{ijs}(h)$ **then**
10: $z^q_{ij\hat{s}}(h) \leftarrow z^q_{ij\hat{s}}(h) + z^q_{ijs}(h) - \lfloor z^q_{ijs}(h) \rfloor$
11: $z^q_{ijs}(h) \leftarrow \lfloor z^q_{ijs}(h) \rfloor$
12: **else**
13: $z^q_{ij\hat{s}}(h) \leftarrow z^q_{ij\hat{s}}(h) - (\lceil z^q_{ijs}(h) \rceil - z^q_{ijs}(h))$
14: $z^q_{ijs}(h) \leftarrow \lceil z^q_{ijs}(h) \rceil$
15: **end if**
16: **end for**
17: **end for**
18: **for** $s \in [S]$ **do**
19: **if** $\hat{x}_{is}(h)$ is fractional **then**
20: $\bar{x}_{is}(h) \leftarrow \lceil \sum_{q\in[Q]} \sum_{j\in[J]} z^q_{ijs}(h) \rceil$
21: **else**
22: $\bar{x}_{is}(h) \leftarrow \hat{x}_{is}(h)$
23: **end if**
24: **end for**
25: **end for**
26: Fix the variable $\{x_{is}(h)\}$ in (8) to $\{\bar{x}_{is}(h)\}$ and solve the new LP (8) again to obtain $\{\bar{y}^q_{ijs}(h)\}$

Therefore, before we add s_k, $\sum_{s\in N\setminus\Theta} g(s)$ must be greater than $T_{k-1}R$. Then, $g(s_k)$ must be greater than the average which is greater than $\frac{T_{k-1}R}{|N|-k+1}$, indicating $T_{k-1} - T_k \geq \frac{T_{k-1}R}{|N|-k+1}$. Thus, we have:

$$\begin{aligned} T_k \leq T_{k-1}(1 - \frac{R}{|N|-k+1}) &\leq |N| \prod_{l=0}^{k-1}(1 - \frac{R}{|N|-l}) \\ &< |N|(1 - \frac{R}{|N|})^k \leq |N| e^{-Rk/|N|} \end{aligned} \tag{10}$$

Taking $k = \frac{|N|}{R}\ln|N|$ yields $T_k < 1$, thus $T_k = 0$. Hence, we prove the lemma. □

Based on Alg. 1, we propose our complete RA algorithm in Alg. 2. If $\hat{x}_{i's}(h')$ is fraction, we add it to a set Ω containing all corresponding $\hat{y}^q_{i'j's}(h')$ (line 4). We leverage Alg. 1 to calculate all the $z^q_{i'j's}(h')$ for $< i', j', h' >$ as well as find the set Θ mentioned above. Then, for all the requests q, we iteratively round all the fractional $z^q_{i'j's}(h')$ until only one $z^q_{i'j'\hat{s}}(h') > 0$ remains fractional with $\hat{s} \in \Theta$ (line 8-16). Meanwhile, we ensure that $\sum_{s\in\Theta^q} z^q_{i'j's}(h')$ remains the same.

Eventually, we reassemble all those $z^q_{i'j's}(h')$ to $\bar{x}_{i's}(h')$ and round it up if it is fractional (line 20). In the end, we fix all the variable $\{x_{is}(h)\}$ in MILP (8) to $\{\bar{x}_{is}(h)\}$. In this way, we transform the original MILP (8) to a new LP (8) which is solved through interior point method and obtain $\{\bar{y}^q_{ijs}(h)\}$.

Theorem 1. *The new LP (8) with $\{x_{is}(h)\}$ fixed to $\{\bar{x}_{is}(h)\}$ has a feasible solution.*

Proof. Without loss of generality, we focus on specific 3-tuple $< i', j', h' >$. In line 8-16, Alg. 2, we ensure that $\sum_{s\in\Theta^q} z^q_{i'j's}(h')$ remains the same for all $q \in [Q]$. Thus, we ensure that $\sum_{s\in[S]}\sum_{q\in[Q]} Y^q \hat{y}^q_{i'j's}(h')$ of total requests still have enough instances to server with the new $\bar{x}_{i's}(h')$. Therefore, we guarantee that the new LP (8) has a feasible solution. □

B. Performance Analysis of RA

Theorem 2. *RA obtains a feasible solution, $\{\bar{x}_{is}(h)\}$ and $\{\bar{y}^q_{ijs}(h)\}$, to the MILP (8) in polynomial time.*

Proof. (Feasibility) Base on theorem 4.2, we observe that $\{\bar{x}_{is}(h)\}$ ensures enough instances to process all the requests, thus renders a feasible solution to the MILP (8).

(Polynomial Runtime) Alg. 1 runs at most $O(QJ + QS^2)$ due to line 1 and the *while* loop in lines 11-15. Therefore, for Alg. 2, the *for* loop from lines 3 to 25 runs at most $O(IJHQ(J+S^2))$. Since solving the LP in line 1 and line 26 takes polynomial time, we arrive at the conclusion that Alg. 2 runs in polynomial time. □

Theorem 3. *The overall cost of the system obtained by RA, C_{all}, is no larger than $C^*_{all} \times \{1 + \frac{|H|c_{sum}}{c_{min}} \min\{\frac{S^2}{S_{min}} \ln S, S^2\}\}$, where C^*_{all} is the minimum objective value of the original MILP (8), $c_{sum} = \sum_{i\in[I]} c_i$, $c_{min} = \min\{c_i, i \in [I]\}$ and $S_{min} = \min\{S^q_{ij}(h), \forall q \in [Q], i \in [I], j \in [J], h \in H\}$.*

Proof. We denote the optimal solution to the original MILP as $x^*_{is}(h)$ and $y^{q*}_{ijs}(h)$. Therefore, we have:

$$
\begin{aligned}
& \frac{C_{all}}{{C_{all}}^*} \\
\leq\ & \frac{\sum_{i\in[I]}\sum_{h\in H}\sum_{s\in[S]} c_i\hat{x}_{is} + \sum_{i\in[I]}\sum_{h\in H}\sum_{s\in[S]} c_i \min\{\frac{S}{S_{min}}\ln S, S\}}{C^*_{all}} \\
\leq\ & 1 + \frac{|H|c_{sum}\min\{\frac{S^2}{S_{min}}\ln S, S^2\}}{\sum_{i\in[I]}\sum_{h\in H}\sum_{s\in[S]} c_i x^*_{is}(h)} \\
\leq\ & 1 + \frac{|H|c_{sum}}{c_{min}}\min\{\frac{S^2}{S_{min}}\ln S, S^2\} \qquad (11)
\end{aligned}
$$

In Alg. 2, for every 2-tuple $< i, h >$, up to $|\Theta|$ $\hat{x}_{is}(h)$ will be round up. And due to lemma 4.1, $|\Theta|$ is upper-bounded by $\frac{S}{S_{min}} \ln S$. Therefore, we have the first inequality. The second inequality is due to $\frac{\sum_{i\in[I]}\sum_{h\in H}\sum_{s\in[S]} c_i\hat{x}_{is}}{\sum_{i\in[I]}\sum_{h\in H}\sum_{s\in[S]} c_i x^*_{is}(h)} \leq 1$ while the third inequality is because at least one instance has to be deployed in the optimal solution. □

V. System Design

A. System Overview

This section presents the system design and implementation of A^2. As illustrated Fig. 3, our system includes three parts.

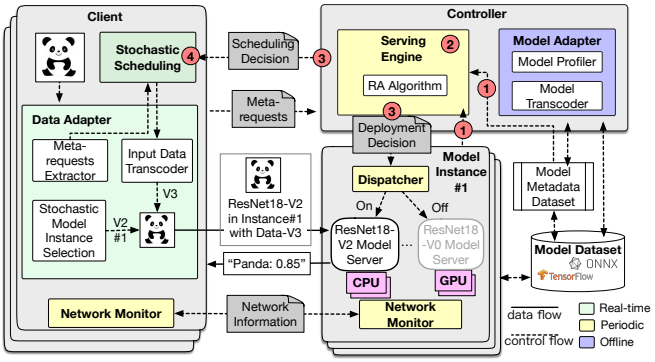


Fig. 3: The key components of A^2 system.

1) Controller.: The controller is the key component of the A^2 system, it has two main modules: *Serving Engine* and *Model Adapter*. The model adapter is triggered when there is a new deep model registered in the system. The model transcoder module "compresses" the deep model using the state-of-art model optimization methods into several model files with different versions stored in the model dataset. Then the model profiler will first extract the metadata of each deep model server, such as average inference latency, FLOPs and batches per second (BPS). In this paper, we use `PROFILING` tools in TensorFlow to measure such model metadata.

The serving engine maintains model instance information and the client-to-server bandwidth history record, by asking clients to report the upload speeds to the model instances they have contacted recently. It receives *meta-requests* from clients in the system. A meta-request contains the ID of the model for the inference, the latency and accuracy requirements, and the client information (e.g., its IP address). The serving engine is running the algorithms in the previous section with the model metadata, and distributes the model deployment decisions to model instances, and responds to a client with the scheduling decision.

2) Client.: Upon receiving the scheduling decision from the controller, a client performs stochastic scheduling based on the scheduling decision. The scheduling decision contains serving configurations (to redirect to a specific model version server in a model instance with a selected data version) and the corresponding fractional value that is used the probability of serving the specific type requests, *e.g.,* with the same location, requirement and deep model. Hence, the client could randomly choose a model server loaded on a specific model file to conduct the inference according to the corresponding probability in the scheduling decision; and compresses the data according to the data version given in the decision and uploads

the compressed data to the model server using `RESTful API` in a real-time manner. For every several (*e.g.,* 10) seconds while the client is not uploading, it still measures the upload speeds to the model instances maintained, and periodically reports the upload speeds to the controller.

3) Model Instance.: A model instance refers to the geo-distributed data center hosts model servers, which receives the deployment decision from the controller and selectively starts the several model servers bound with different hardware, for instance, in this paper, we use `TENSORFLOW SERVING` docker to host model version file (*e.g.,* the ResNet18 model-server-2 file) with one CPU core. When the model server receives a request from the client, it may wait for more requests to form a requests batch for serving parallelly using the adaptive batching technique, then returns the serving results to the clients. Notice that, the idle model instance could also fine-tune the model or compress the model, the validation data collected from clients (the result of origin model regarded as the ground-truth [20]) into different versions and update the model metadata of inference such as throughput (data samples/second) and inference latency per input data sample of each model server.

B. A^2 Workflow

A^2 periodically updates the updates scheduling and deployment decisions, the A^2 as a finite-state machine which has four stages:

1) Initialization: A^2 needs to collect model instance information and model metadata, then set up the initial settings for the RA algorithm based on the model metadata. At the beginning of the system are launched, because lack of the information to predict the workloads of requests and network conditions, the Controller will randomly generate the workloads of requests and network conditions to start. Besides, the model instances will launch the model servers using the cold-start deploy policy (*e.g.,* random or all original versions) and the clients will also conduct the cold-start scheduling policy (*e.g.,* random or default inference configuration). After the initialization, the first time-slot will start.

2) Policy update: Then, at the beginning of each time-slot, the RA algorithm is conducted based on the hyper-parameters and outputs the deployment decision for model instances and scheduling decisions for clients.

3) Decision dispatch and update: Once the model instances receive the deployment decision, it will re-launch the model servers based on the deployment decision. When clients receive the scheduling decision, the clients will update the scheduling decision table stored.

4) Real-time request scheduling and inference: Finally, a client provides the real-time request scheduling and inference, by randomly selecting an inference configuration based on the probability in a scheduling decision table. When the next time-slot arrives, A^2 goes back to stage 2) to update the policy. In this paper, the duration of a one time-slot is set to 1 minute.

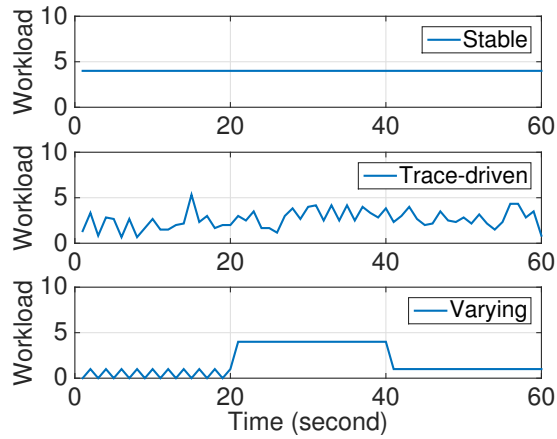


Fig. 4: Illustration of a one-minute sample of the three types of workloads used in our experiments.

VI. Evaluation

A. Experiment Setup

To evaluate our solution, we conduct trace-driven experiments on 13 geo-distributed VM instances allocated on Amazon EC2, spanning 6 sites (Singapore, Tokyo, Mumbai, Oregon, Ohio, and Sydney), each of which represents a region as in our design. All instances run Deep Learning AMI (Ubuntu) Version 23.1 with Ubuntu 16.0 (4.12.9 kernel), and have up to 10Gbps networking capacity. In each region, we allocate a `m5.2xlarge` EC2 instance with 32 GiB of Memory and 8 vCPUs as a *model instance*, and a `m5.2xlarge` EC2 instance with 16 GiB of Memory and 4 vCPUs as a *client instance*. Each model instance and each client instance run several model servers and clients. The controller is deployed on a `m5.xlarge` instance at Singapore. We use the models with four versions and sixteen data versions as in Sec. § II. We use the `Traffic Control (TC)` tool of linux to control the bandwidth between Amazon EC2. We use the FLOPs of each model server i as the serving cost in the experiments (The serving cost could be any reasonable metrics chosen by the system users to represent the monetary cost as described before).

Client Requests: On each client instance, we simultaneously execute 3 client processes (i.e., 3 Linux processes). Each client process generates requests with the requesting workload as below, latency and accuracy requirements, and it sends the requests to different model servers according to the scheduling decision received from the controller. The workloads at the clients are as follows.

Workloads: We use traces and models to drive clients' inference workloads, as follows. 1) Trace-driven Workload: we use real-world traces collected from a mobile video provider to drive the inference requests, as machine learning services have been highly incorporated into such multimedia applications. The dataset spans 2 weeks and covers 2 million users watching 0.3 million unique videos. 2) Varying Workload: we generate the second workload such that the number of requests varies

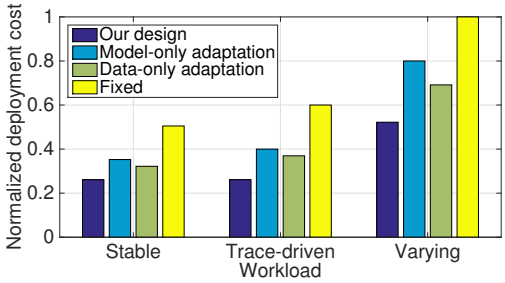


Fig. 5: Deployment cost: normalized by comparing to the $Fixed$ scheme.

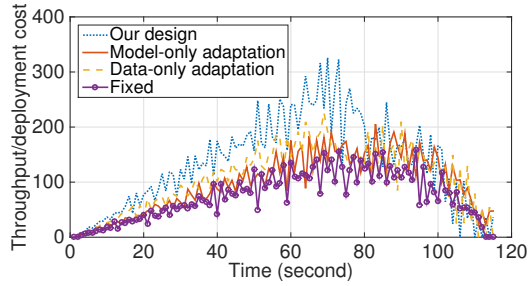


Fig. 6: Thoughput comparison

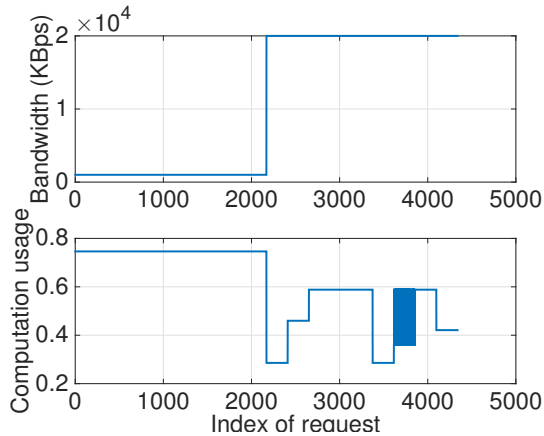


Fig. 7: Bandwidth and computational resource usages over time.

significantly over time. 3) Stable Workload: on the other hand, we also generate a third type of workload where the number of requests over time is stable at a relatively high level. Fig. 4 illustrates a one-minute sample of the above workloads. In the following experiments, the workloads will be mapped to the number of requests generated by a client, and the probability of a request to a particular model following a zipf like distribution among all the models.

Baselines: We compare our design with the following baselines. The default rounding scheme is the RA algorithm.

- **Fix**: The inference is based on the fixed model and data, i.e., the same model serves clients that upload the same version of data. This strategy is used by the conventional NVIDIA TensorRT Inference Server and TensorFlow Serving, as well as new frameworks including Clipper [5], SageMaker and CloudML and Clockwork [16].
- **One-Adaptation**: Since there is no existing study that provides a joint model and data adaptation for model serving, we also compare our design with naive adaptations: 1) Inference serving with only adaptive model versions (Model-Only), *e.g.,* INFaas [6] and OSFA [7] and 2) Inference serving with only adaptive data versions (Data-only) *e.g.,* AWStream [17].

Default system configuration: By default, β^q for the latency requirement for each request is assigned randomly in $[1.5, 5]$ seconds, the accuracy requirement α_q for each request is uniformly sampled from $[0.75, 0.9]$, and the upload capacity between a client instance and a server instance is set to be $2,000$KBps. We set $45,000$KBps as the bandwidth capacity of each model server running a particular model version. We use the same image with a size of 100KB for all clients to upload to the model servers.

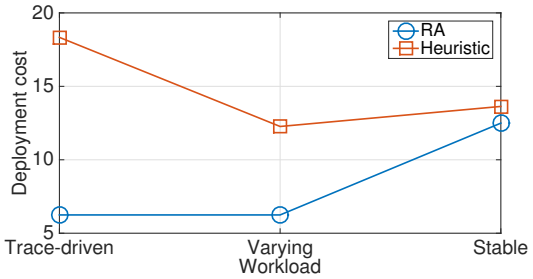
B. Comparative Results

1) Deployment Cost: We first compare the deployment cost, cost calculated using Eq. 7 in Sec. § III-C, under different solutions. In Fig. 5, we plot the deployment cost normalized as a reduction against the $Fixed$ baseline, which has the highest deployment cost among all the strategies. In this figure, we observe that our solution achieves the best deployment cost saving, and model-only and data-only adaptations have similar performance on deployment cost saving. Compared to $Fixed$ scheme, our design significantly saves up to 50% deployment cost, especially when the workload is varying over time. Compared to model-only or data-only adaptation solutions, our design can still save the deployment cost by around 18%. The results confirm that our design can truly benefit by jointly considering the diverse latency and accuracy requirements.

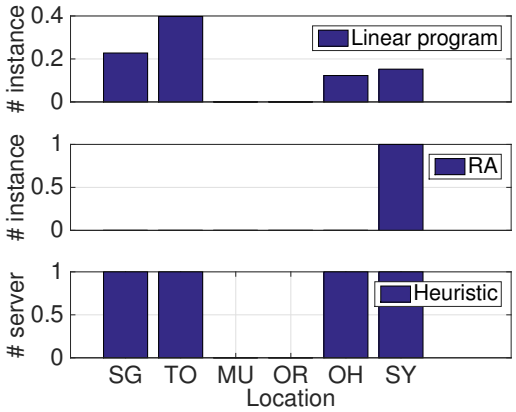
2) Throughput per Deployment Cost: Next, we study how our design performs under varying workloads. In Fig. 6, we plot the request serving throughputs per deployment cost with different solutions over time, under the varying workload. We observe our design outperforms other baselines significantly: 1) Our design achieves a significant improvement of throughput per deployment cost by 79% and 28% as compared to the fixed solution and model-only adaptation solution, respectively. 2) When there are more requests generated by the clients, our design has even larger throughput per deployment cost, compared to lower workload, which verifies that our design can adaptively make use of small models and/or data versions when there are more concurrent requests.

C. Effectiveness of Our Design

1) Resource Tradeoff: We show that our design is truly able to take advantage of trading off the bandwidth and computational resources. In this experiment, we set all requests to have the same accuracy requirement of 0.84 and latency requirement within 0.5 second. In Fig. 7, by controlling the bandwidth capacity, i.e., letting the bandwidth raise from



(a) Deployment cost under different rounding algorithms.



(b) Number of model servers to allocate under different algorithms

Fig. 8: Impact of different algorithms.

100KBps to 2,000KBps, at the 2170-th request, we investigate the computational resource usage, which is defined as a normalization of computation consumption ratio compared with the total resource computation. A higher computation usage means the requests take more computational resources. We observe that when the bandwidth capacity is high, the solution tends to use more computational resources to provide a similar inference performance, and vice versa. The results suggest our design is able to jointly utilize the two types of resources.

2) Effectiveness of Rounding-based Approximation: We study the impact of the rounding algorithm on the deployment cost. In Fig. 8, we plot the total deployment cost for RA and the heuristic solution (Rounding up all the fractional value), respectively, under different workloads. We observe that our rounding-based approximation has a significantly lower deployment cost, with a cost saving by around 50%, and especially for the trace-driven and varying workloads, indicating that our solution is effective to handle more dynamical inference workloads.

We further explore the reason behind this advantage. In Fig. 8(b), we show the number of model servers allocated by different algorithms, including the naive linear program, our rounding-based approximation and the heuristic solution.

We observe that after the linear program generates the fractional deployment decision, i.e., 0.22 for Singapore, 0.39 for Tokoy, 0.12 for Ohio, and 0.15 for Sydney, our RA only "transfers" the fractional deployment decision to only allocating 1 model server in Sydney; while the heuristic scheme allocates 1 model server for Singapore, Tokyo, Ohio, and Sydney, respectively. As a result, our design has saved many deployment costs in similar scenarios with small fractional LP-generated allocations.

3) Analysis on Latency Violation: As the deployment cost saving in our design is partially due to the adaptive request scheduling, we further study the impact of our solution on the inference latency experienced by users practically. In Table II, we use the latency violation rate to denote the fraction of inference requests that are not served within their latency requirements. Notice that there can be latency violation even we have the latency constraint in our optimization and algorithm design, because of the dynamics in network conditions and the decision delay. In this table, we also calculate the mean and standard deviation of the request postpones (i.e., the delay experienced after allowed latency requirement), normalized by their allowed latency requirements. A minus postpone indicates that a request is served within the latency required. We observe that our design has quite similar latency violation performance compared to other baselines, and they all satisfy the P99 latency requirement.

VII. Related Work

Inference Serving. Machine learning services have been widely incorporated into a variety of online services and applications. A number of inference systems have been proposed for *serving* the machine learning models. TensorFlow Serving [21] and Clipper [5] and Clockwork [16] are the VM-based methods, the models are hosted in containers or dockers which are analogous to webservers. They focus on model management (*e.g.,* model pre-loading), latency-throughput trade-offs (*e.g.,* adaptive batching, multiple model serving). While the VM-based methods improve the inference performance, they still need to manage hardware resources manually. Thus, Cloud venders provide ML as a Service (MLaaS), with offerings from Google [2], Amazon [1] and Microsoft [3] and others. These services run on the cloud, and expose an API to a DNN model trained on uploaded datasets, easing the deployment complexity, as users do not have to manage specific resources or interact with underlying VMs manually. When the request workload increases, their internal resources will be auto-scaled as [22]. INFaaS [6] dynamically scale the serving dockers with various model variants, which differ in computation cost, memory demand, inference latency, and achievable accuracy.

Due to the explosive increase of requests to the machine learning services, adaptive solutions have been investigated so as to reduce the consumption of computation with a still satisfactory quality of service. Taking the insight of accuracy-latency trade-off, OSFA [7] allows consumers to choose tolerance tiers in Machine Learning as a Service (MLaaS) platforms which enable consumers to sacrifice the service's result quality by using multiple model versions, to improve

	Latency violation rate				Mean and standard deviation of request postpones (normalized)			
	Our Design	Model-only adaptation	Data-only adaptation	Fixed	Our Design	Model-only adaptation	Data-only adaptation	Fixed
Stable	0.80%	0.58%	0.69%	0.71%	46% ±33%	42% ±0.49	37% ±35%	37% ±29%
Trace-driven	0.27%	0.07%	0.07%	0.41%	7% ±4.8%	0.3% ±0	4% ±0	29% ±23%
Varying	0.15%	0.25%	0.35%	0.30%	11% ±7%	20% ±80%	8% ±9%	7% ±3%

TABLE II: Latency violation rate, and mean and standard deviation of request postpones.

other aspects of the service, such as the service response time and invocation cost. INFaaS [6] utilizes computation resources finer granularly: it dynamically scales the serving dockers with various model variants, which differ in computation cost, memory demand, inference latency, and achievable accuracy. However, these studies only focus on the efficient utilization of computational resources and do not address the trade-off between bandwidth and computation resources.

Adaptive Serving. There are many adaptive serving applications in a specific service. For computation offloading, Han et al. [23] proposed MCDNN, which is able to perform approximation execution for existing models, with resource consumption as its constraints. Bateni et al. [24] proposed a timing-predictable runtime system that is able to guarantee deadlines of DNN workloads via layer-wise network approximation. Recently, Fang *et al.* [25] has proposed a scheduling framework for multi-tenant mobile offload systems serving computer vision algorithms and DL workloads. For video analysis, Zhang et al. [11] proposed VideoStorm, which performs live video analytics with approximation and delay-tolerance model execution. Jiang et al. [20] proposed Chameleon that adaptively selects the video encoding configuration for inference with prefixed rules. Kang et al. [26] propose NoScope, which particularly focuses on optimizing neural network queries for video. [27] propose JCAB to adaptively choose the video configuration and shared bandwidth allocation policy. Lee et al. [28] introduced a dynamic subnetwork constructions for DNNs. While these studies are not generally inference serving system solution, without considering of the end-to-end inference latency or focusing on the specific data or the specific model.

To the best of our knowledge, our study is the first to explore the new design space enabled by jointly considering the adaptation of both deep learning models and data versions.

VIII. Conclusion

In this paper, a new adaptive deep model inference serving system is designed to provide guarantees of scalability, cost efficiency and satisfy user requirements. By taking the insight that to achieve the same accuracy level for a deep learning model, one can choose different combinations of data versions and model versions, we design a new inference framework called A^2, joint model and data adaptation for deploying inference services. To minimize the total serving cost while meeting latency and accuracy demand for the serving request, we formulate the model deployment and request serving problem as a minimization of the system cost while satisfying latency and accuracy constraints of requests, and design a RA rounding scheme, by efficiently finding the overlap of model version instance satisfying the accuracy and latency constraints. Finally, a system prototype is designed and deployed to Amazon Cloud to evaluate and demonstrate the effectiveness and efficiency of our proposed design, that A^2 significantly reduces the system cost up to 50% against existing solutions, by dynamically trading off bandwidth and computation resources, while achieving clients constraints.

REFERENCES

[1] https://aws.amazon.com/cn/machine-learning/.

[2] https://cloud.google.com/vision/.

[3] https://www.microsoft.com/cognitive-services/.

[4] C. Olston, N. Fiedel, K. Gorovoy, J. Harmsen, L. Lao, F. Li, V. Rajashekhar, S. Ramesh, and J. Soyke, "TensorFlow-Serving: Flexible, High-Performance ML Serving," *arXiv preprint*, 2017.

[5] D. Crankshaw, X. Wang, G. Zhou, M. J. Franklin, J. E. Gonzalez, and I. Stoica, "Clipper: A low-latency online prediction serving system," in *NSDI*, 2017, pp. 613–627.

[6] F. Romero, Q. Li, N. J. Yadwadkar, and C. Kozyrakis, "INFaaS: Managed & Model-less Inference Serving," *arXiv preprint*, 2019. [Online]. Available: http://arxiv.org/abs/1905.13348

[7] M. Halpern, B. Boroujerdian, T. Mummert, E. Duesterwald, and V. Janapa Reddi, "One Size Does Not Fit All: Quantifying and Exposing the Accuracy-Latency Trade-Off in Machine Learning Cloud Service APIs via Tolerance Tiers," in *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, mar 2019, pp. 34–47. [Online]. Available: https://ieeexplore.ieee.org/document/8695638/

[8] P. Mattson, V. J. Reddi, C. Cheng, C. Coleman, G. Diamos, D. Kanter, P. Micikevicius, D. Patterson, G. Schmuelling, H. Tang *et al.*, "Mlperf: An industry standard benchmark suite for machine learning performance," *IEEE Micro*, vol. 40, no. 2, pp. 8–16, 2020.

[9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[10] K. Hazelwood, S. Bird, D. Brooks, S. Chintala, U. Diril, D. Dzhulgakov, M. Fawzy, B. Jia, Y. Jia, A. Kalro *et al.*, "Applied machine learning at facebook: A datacenter infrastructure perspective," in *High Performance Computer Architecture (HPCA), 2018 IEEE International Symposium on*. IEEE, 2018, pp. 620–629.

[11] H. Zhang, G. Ananthanarayanan, P. Bodik, M. Philipose, P. Bahl, and M. J. Freedman, "Live video analytics at scale with approximation and delay-tolerance," in *NSDI*, vol. 9, 2017, p. 1.

[12] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, "Pruning filters for efficient convnets," *arXiv preprint arXiv:1608.08710*, 2016.

[13] J. Wu, Y. Zhang, H. Bai, H. ZhongLatency, T. C. J. Hou, W. Liu, and J. Huang, "Pocketflow: An automated framework for compressing and accelerating deep neural networks," in *Advances in Neural Information Processing Systems (NIPS), Workshop on Compact Deep Neural Networks with Industrial Applications*, 2018.

[14] X. Nan, Y. He, and L. Guan, "Optimal resource allocation for multimedia cloud in priority service scheme," in *2012 IEEE International Symposium on Circuits and Systems*. IEEE, 2012, pp. 1111–1114.

[15] W. Wang, B. Li, and B. Liang, "Dominant resource fairness in cloud computing systems with heterogeneous servers," in *IEEE INFOCOM 2014-IEEE Conference on Computer Communications*. IEEE, 2014, pp. 583–591.

[16] A. Gujarati, R. Karimi, S. Alzayat, W. Hao, A. Kaufmann, Y. Vigfusson, and J. Mace, "Serving dnns like clockwork: Performance predictability from the bottom up," in *14th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 20)*, 2020, pp. 443–462.

[17] B. Zhang, X. Jin, S. Ratnasamy, J. Wawrzynek, E. A. Lee, and E.-W. A. Lee, "AWStream: Adaptive Wide-Area Streaming Analytics," 2018. [Online]. Available: https://doi.org/10.1145/3230543.3230554

[18] N. J. Yadwadkar, F. Romero, Q. Li, and C. Kozyrakis, "A Case for Managed and Model-less Inference Serving," in *Proceedings of the Workshop on Hot Topics in Operating Systems - HotOS '19*. New York, New York, USA: ACM Press, 2019, pp. 184–191.

[19] S. Boyd and L. Vandenberghe, *Convex optimization*. Cambridge University Press, 2004.

[20] J. Jiang, G. Ananthanarayanan, P. Bodik, S. Sen, and I. Stoica, "Chameleon: scalable adaptation of video analytics," in *Proc. of SIGCOMM*. ACM, 2018, pp. 253–266.

[21] C. Olston, N. Fiedel, K. Gorovoy, J. Harmsen, L. Lao, F. Li, V. Rajashekhar, S. Ramesh, and J. Soyke, "Tensorflow-serving: Flexible, high-performance ml serving," *arXiv preprint arXiv:1712.06139*, 2017.

[22] A. Gujarati, S. Elnikety, Y. He, K. S. McKinley, and B. B. Brandenburg, "Swayam: Distributed Autoscaling to Meet SLAs of Machine Learning Inference Services with Resource Efficienc," in *Proceedings of the 18th ACM/IFIP/USENIX Middleware Conference on - Middleware '17*. New York, New York, USA: ACM Press, 2017, pp. 109–120.

[23] S. Han, H. Shen, M. Philipose, S. Agarwal, A. Wolman, and A. Krishnamurthy, "MCDNN: An Approximation-Based Execution Framework for Deep Stream Processing Under Resource Constraints Seungyeop," in *Proceedings of the 14th Annual International Conference on Mobile Systems, Applications, and Services - MobiSys '16*. New York, New York, USA: ACM Press, 2016, pp. 123–136.

[24] S. Bateni and C. Liu, "Apnet: Approximation-aware real-time neural network," in *2018 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2018, pp. 67–79.

[25] Z. Fang, J.-H. Lin, M. B. Srivastava, and R. K. Gupta, "Multi-tenant mobile offloading systems for real-time computer vision applications," in *Proceedings of the 20th International Conference on Distributed Computing and Networking - ICDCN '19*. New York, New York, USA: ACM Press, 2019, pp. 21–30.

[26] D. Kang, J. Emmons, F. Abuzaid, P. Bailis, and M. Zaharia, "NoScope: Optimizing Neural Network Queries over Video at Scale," *Proceedings of the VLDB Endowment*, 2017.

[27] C. Wang, S. Zhang, Y. Chen, Z. Qian, J. Wu, and M. Xiao, "Joint configuration adaptation and bandwidth allocation for edge-based real-time video analytics," in *Proc. of the 39th IEEE International Conference on Computer Communications (INFOCOM 2020)*, 2020.

[28] S. Lee and S. Nirjon, "Subflow: A dynamic induced-subgraph strategy toward real-time dnn inference and training," in *2020 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 2020, pp. 15–29.